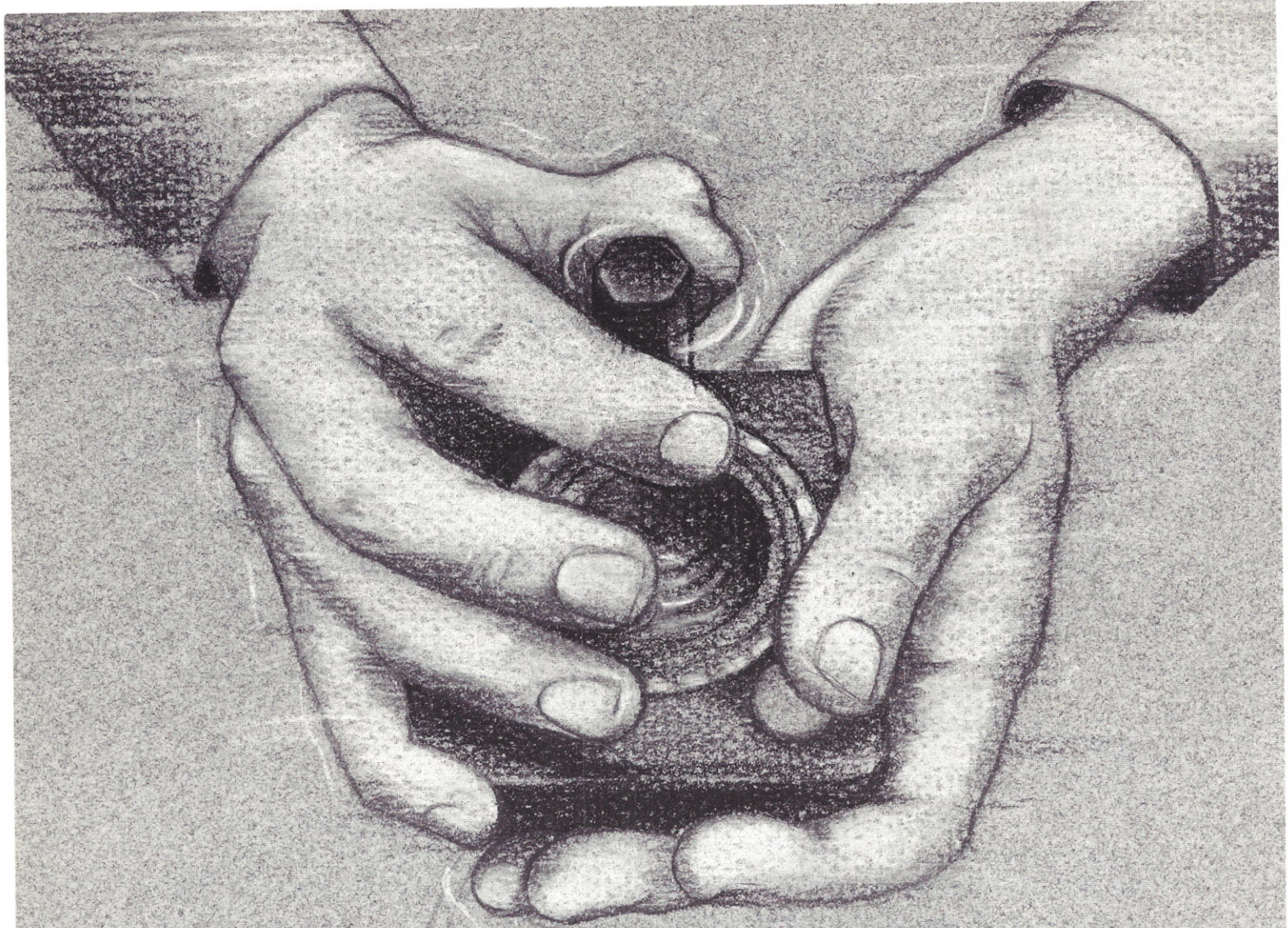


INTERFACING EXTERNAL DEVICES

add a game joystick to the Osborne 1

Kelly Smith

Editor's note—The subject of external devices talking to and interacting with a personal computer is one of great interest. Today, many applications exist for scientific instruments to work side by side with a computer. For example, in the medical profession, a microcomputer can be used to monitor a patient's vital signs and take appropriate steps when the measurements appear abnormal. In the engineering field, computers routinely monitor and control temperature, light intensity, valves, and electrical current flow, to name a few.

A game joystick is not a scientific instrument, but it does represent an external device that can produce certain responses on a host computer. It's also a product that is inexpensive, sold just about everywhere, and simple to connect and operate. Keep in mind that the theories and principles behind interfacing a device as simple as a joystick can be carried over to any number of applications. In that regard, Kelly Smith's article provides an excellent foundation towards the understanding of the Osborne 1 as a tool that can interact with its environment. In this first of two parts, Kelly shows you how to:

- *Connect and wire an Atari joystick*
- *Create an "Etch-a-Sketch" assembly program that tests the joystick*
- *Write a program that can be used within any MBASIC application requiring a joystick interface*

In next month's Portable Companion, we'll publish Kelly's educational and informative SPACEWAR program, which utilizes the joystick.—bb

Osborne 1 IEEE-488 to Joystick, Interface Adaptor Cable

Time now to gain a little hardware experience. The parts required for the interface adaptor cable can be purchased at an electronic supply house or your local Radio Shack retailer. You will need:

- one DB-9 Male connector (solder type)
- one 26 pin (13 pin dual row) card edge/flat ribbon connector that mates with the IEEE-488 connector on your Osborne
- about 7 inches of 12 conductor flat ribbon cable (14 conductor is standard, just strip off 2 conductors to make 12) is also needed
- an Atari game joystick (about $15.00)

(See the note at the end of this article for information on obtaining a ready-made game diskette, cable and joystick.)

The external IEEE-488 connector (labeled "IEEE-488") pin assignments and all relevant data signals appear as shown in Figure 1 when looking at the front of your Osborne 1 computer:

```
                                                    D3   D2   D1   D0
 25   23   21   19   17   15   13   11    9        7    5    3    1
(left)                                                          (right)
 26   24   22   20   18   16   14   12   10        8    6    4    2
                                  GND             D7   D6   D5   D4
```

Figure 1

The DB-9 Male connector (it mates with the Atari joystick) pin assignments and data signals appear as shown in Figure 2 (back view, solder pin side). Note that only signals DØ to D4 and sample signal (GND) are used when wiring the joystick interface cable:

```
        D2    D3    D1    D0
  5      4     3     2     1

     9     8     7     6
         GND         D4
```

Figure 2

DØ to D7 comprise the eight possible data bits for the joystick data byte. Signal GND is a ground return that may be sampled as a switch closure in any one of five possible switch positions (Up, Down, Right, Left, Fire) of the game joystick. Signal name, connector pin numbers and functions are assigned as shown in Figure 3:

Signal	IEEE-488 Pin #	Joy Stick Pin #	Function
Data 0	1	1	Move Up
Data 1	3	2	Move Down
Data 2	5	4	Move Right
Data 3	7	3	Move Left
Data 4	2	6	Fire Button
Data 5	4	(not used)	(spare)
Data 6	6	(not used)	(spare)
Data 7	8	(not used)	(spare)
Ground	12	8	Sample Signal

Figure 3

If assembled correctly, the flat ribbon cable should exit UP from the IEEE-488 connector, and then loop down (for strain relief) to go under the case of the Osborne 1 (if you rest its edge on the keyboard case for better screen visibility). The cable is then neatly out of the way of the keyboard. Also, note that other game joysticks can work, and in fact some offer as many as eight different switch closures. If you feel so inclined, experiment. I chose just one of the easy possibilities, because it was convenient to modify SPACEWAR (my game appearing in next month's issue), that I had previously written to use the Osborne 1 numeric keypad for position control.

Keep in mind that this does not even have to be a game joystick. Any switch closure to the ground pin can be sampled from a variety of sources—magnetic switches, mercury 'tilt' switches, bi-metallic temperature switches, etc., etc. (I am hinting that a home control/security system would be VERY EASY to do... consider the possibilities). Just one word of caution: these switches must have full ISOLATION from any source of external electrical power.

DO NOT (for example) hook your Osborne 1 up to your front door bell/buzzer switch. It is guaranteed "Zorch-City" as you fry the electronics in your $1,795 Osborne the first time someone rings the bell. "Zorch" is one computer game that will cost you more than a quarter to play.

A Simple "Etch-a-Sketch" Checkout Program

Now, we need to create a simple program that checks all possible switch combinations on our newly-made interface adaptor cable by graphically displaying the position of a "pointer" on the screen display of the Osborne. It's easy to make an electronic "Etch-a Sketch" using the entire 4096 character positions of the display. Edit the following assembly language program with WordStar (as a Non-document), assemble and load it (refer to your *User's Reference Guide* for the discussions of ASM and LOAD).

Now, connect your interface adaptor cable to the IEEE-488 connector on your Osborne 1 front panel, boot your system with a diskette that has been SETUP with Auto-Scroll set to OFF, and locate the pointer that is in the middle of the display screen. Use the CTRL (Control) and Left Arrow key to scroll the screen display until you locate the pointer. Now the fun begins!

If your interface adaptor cable is wired properly, you should be able to write all over the screen in any direction. Notice that any point that is crossed-over suddenly flips to a brighter (or dimmer) display position. Swirling the joystick 360 degrees will produce some very interesting patterns. Press the button on the joystick at any time to entirely clear the display screen.

The code at label "begin" initially set the register pair H and L to a value that "points" to the center of the Osborne 1 screen memory. This pointer [HL] is then saved (push h) on the push-down stack to preserve its content from the certain destruction of the use of [HL] in the routines "delay" and subsequent "call 0e154h" (the IEEE-488 input parallel poll routine).

The call to a delay subroutine is required only if you want to draw things on the screen with the speed of normal human response time (drawing circles, angles, etc.). Remove the call to the delay subroutine and the response to any joystick movement is instantaneous — filling the screen with patterns in a flash. The actual "workhorse" that reads the joystick is all taken care of for us in the IEEE-488 input parallel poll routine. A call to that address simply returns the bit pattern (in the [A] register) corresponding to the mechanical switch closures within the joystick.

The pointer [HL] is recovered from the stack (pop h), and the joystick data in [A] is then compared to see which (if any) switch position(s) have been closed. Also, a corresponding offset value is pre-loaded into registers [DE] in anticipation that the value will be added to [HL] from [DE] (in the "wrt$crt" subroutine that positions the pointer in the screen memory display). The main program loops through all eight possible switch closure positions (refer to Figure 4), checking for the next direction to move the pointer by adding an offset value to the current position in screen memory ... whew!

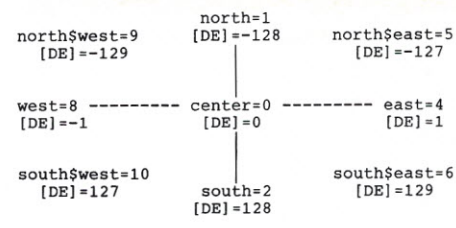


Figure 4

Electronic Etch-a Sketch

```
        org     100h    ; start of CP/M transient program area
;
begin:  lxi     h,0f5bfh; center of 128 X 32 osborne 1 screen
start:  push    h       ; save pointer to video screen position
        call    delay   ; if no delay, this fills the screen FAST
        call    0e154h  ; 1.4 bios call to IEEE-488 input parallel poll
;
;note--call 0e554h for 1.3 bios (single density) Osbornes.
;
        pop     h       ; get pointer to video screen position
;
center:                 ; stay where you are...
;
        cpi     0       ; if zero, stick is center position
        lxi     d,0
        jz      wrt$crt
;
north:                  ; up one line
;
        cpi     1       ; "...to Alaska"
        lxi     d,-128
        jz      wrt$crt
;
south:                  ; down one line
;
        cpi     2       ; "...to Tijuana"
        lxi     d,128
        jz      wrt$crt
;
east:                   ; over one character position
;
        cpi     4       ; "...to Jersey"
        lxi     d,1
        jz      wrt$crt
;
north$east:             ; back one line, plus one character position
;
        cpi     5       ; a nor'easters a'blowin mate...
        lxi     d,-127
        jz      wrt$crt
;
south$east:             ; over one line, plus one character position
;
        cpi     6       ; off to EPCOT Center...
        lxi     d,129
        jz      wrt$crt
;
west:                   ; back one character position
;
        cpi     8       ; "go west young joy stick"
        lxi     d,-1
        jz      wrt$crt
;
north$west:             ; back one line, minus one character position
;
        cpi     9       ; Mounties?
        lxi     d,-129
        jz      wrt$crt
;
south$west:             ; over one line, minus one character position
;
        cpi     0ah
lxi     d,127
        jz      wrt$crt
;
;
        cpi     10h     ; "on a clear disk, you can seek forever"
        cz      clear
;
        jmp     start   ; loop back and sample joy stick again
;
;
clear:  mvi     e,'Z'-40h       ; somehow this all becomes much
        mvi     c,2             ; clearer after a 6-pack of 'Oly'
        push    h
        call    5
        pop     h
        ret             ; as if by magic, the screen is cleared
;
wrt$crt:                ; write the boob tube...
;
        dad     d       ; add position offset to [HL]
        mov     a,h
        cpi     0f0h    ; trying to crash below screen memory?
        jc      begin   ; if so, just go back screen center
        mov     a,m     ; been here before?
        cpi     07fh
        jnz     block   ; if not, drop a block pattern on the screen
        mvi     m,'V'-40h       ; flip this position
        jmp     start   ; loop back and sample joy stick again
block:  mvi     m,07fh  ; make a block pattern
        jmp     start   ; loop back and sample joy stick again
delay:  lxi     h,800h  ; delay awhile, to keep from moving to fast
loop:   dcx     h
        mov     a,h
        ora     l
        jnz     loop
        ret
;
        end
```

Where do these values come from? Well, if we examine the original layout for the assignment of IEEE-488 data signals DØ to D7 (and we throw in a little binary to decimal conversion), we find that each data bit position has a corresponding decimal value, as shown in Figure 5.

```
Data Bit #:     D7    D6    D5    D4    D3    D2    D1    D0
Decimal Value: 128    64    32    16    8     4     2     1
```

Figure 5

The "real" binary ones-and-zeros bit patterns that come from the IEEE-488 interface are then translated directly from bit number positions to decimal value, and finally, to direction as shown in Figure 6.

bits #'s 76543210	direction	decimal value	screen offset
00000000	center	0	0
00000001	north	1	-128
00000010	south	2	+128
00000100	east	4	+1
00000101	north east	5	-127
00000110	south east	6	+129
00001000	west	8	-1
00001001	north west	9	-129
00001010	south west	10	+127
00010000	fire button	16	(clear screen)

Figure 6

Notice that north east, south east, north west, and south west are the sum of the binary values (actually two simultaneous switch closures)—combinations such as 3 and 7 are not mechanically possible within the joystick. The only exception to this is bit position 4 (decimal value 16) that I chose to use as the "Fire Button" (more on that in Part Two). As programmed, you MUST return to center position on the joystick to detect the closure of the 'Fire Button' ONLY. Can you figure out how to fix this? If you have a joystick with three more switches, they could just as well be assigned to bit positions 5, 6, and 7 and "sampled-in-software" accordingly.

High Level to Low Level IEEE-488 Interfacing

All of the IEEE-488 interfacing was done very simply for us with just a "call" to a predefined high level subroutine built-in to your Osborne 1 computer. There are times, however, when you must access certain portions of a computer at the very basic or lowest level of the computer—now we get into the very soul of the machine.

The IEEE-488 Bus implementation is really just a parallel data port controller that can be used for a variety of interfacing possibilities that require byte-wide (8 bits at a time) data. This controller is known as a 6821 PIA (Peripheral Interface Adaptor), and may be programmed in a variety of different input/output control modes. We are going to program it at the low level in assembly language for use in the joystick control game called SPACEWAR to be introduced next time around.

First the low level programming: Edit the assembly language code shown below with WordStar (yes, Non-document), as a file called JOYSTIK.ASM—assemble it, but DO NOT load it—we need to modify the .HEX file that is generated from the assembly, in the various steps that follow on the next page.

Joystik. ASM

```
; routine to read an Atari joystick via the 6821 PIA/Parallel port
;
porta0  equ     2900h   ; 6821 pia port-a data/direction register
porta1  equ     2901h   ; 6821 pia port-a control register
portb2  equ     2902h   ; 6821 pia port-b data/direction register
portb3  equ     2903h   ; 6821 pia port-b control register
flagpos equ     0ef08h  ; ram/rom flag position address: 0=rom, 1=ram
;
true    equ     -1      ; define true
false   equ     not true; define false
;
; +++ only one of the following 'versions' may be set true for assembly +++
;
; Note: if in doubt, press reset and look at the monitor version number
;
ver2    equ     false   ; version 1.2 system
ver3    equ     false   ; version 1.3 system
ver4    equ     true    ; version 1.4 system
;
        if      ver2
        org     0ee80h
        endif

        if      ver3
        org     0ed80h
        endif

        if      ver4
        org     0ea80h
        endif
;
sdata   db      0       ; save joystick data here (initially no direction)
;
; subroutine to read joystick data and save it in previous location
;
rstick: di              ; quiet please
        out     0       ; rom on/ram off
        xra     a
        sta     flagpos ; inform the system that rom is on
        lda     porta0  ; read the IEE-488/Parallel port
        mov     b,a     ; save data in register B temporarily
        out     1       ; ram on/rom off
        mvi     a,1     ; inform the system that ram is on
        sta     flagpos
        ei              ; you may talk now
        mov     a,b     ; back from exile
        ani     01fh    ; mask for data bits D0 to D4 only
        sta     sdata   ; save as  joystick data to be read later
        ret             ; return to calling routine (probably in BASIC)
;
; subroutine to initialize 6821 PIA/Parallel port for joystick control
;
istick: di              ; quiet please
        out     0       ; rom on/ram off
        xra     a
sta     flagpos ; inform the system that rom is on
sta     porta1  ; clear control and data ports to float the 6821 PIA
sta     porta0
mvi     a,4     ; set control for peripheral register-a direction in
sta     porta1
xra     a
sta     portb3  ; float register-b port, for now
cma             ; set for direct data out
sta     portb2
mvi     a,4     ; set control for peripheral register-a direction in
sta     portb3
mvi     a,0ffh  ; set for direct data out
sta     portb2
out     1       ; ram on/rom off
mvi     a,1     ; inform the system that we are back in ram
sta     flagpos
ei              ; you may talk now
ret

end
```

1 JOYSTIK.HEX after assembly of JOYSTIK.ASM

After assembly of JOYSTIK. ASM, two files will be created. They are JOYSTIK.PRN, and JOYSTIK.HEX. Enter TYPE JOYSTIK.HEX and your keyboard RETURN key then compare it to the following explanation of what a file of type .HEX is comprised of (if you have a printer. Also enter CTRL-P (press the CTRL key and the P key together) just before the RETURN key to get a listing (entering CTRL-P again, will turn the printer output off):

```
   start of record identifier (colon character)
    1 byte record byte count (10 Hex/16 Decimal)
     2 byte load address (EA80 Hex, for 1st record)
      1 byte record type (read)
            16 bytes data ______|  |___ 1 byte checksum
:10EA800000F3D300AF3208EF3A002947D3013E0127 <-- 1st record
:10EA90003208EFFB78E61F3280EAC9F3D300AF32C1 <-- 2nd record
:10EAA00008EF3201293200293E04320129AF320332 <-- 3rd record
:10EAB000292F3202293E043203293EFF320229D390 <-- 4th record
:08EAC000013E013208EFFBC91D                 <-- 5th record
:0000000000     <-- 6th (and final) record and end of file
```

Now, enter TYPE JOYSTIK.PRN and note the address of 'sdata' (joystick data, address EA80,1.4 bios), 'rstick' (read joystick subroutine, address EA81), 'istick' (initialize joystick subroutine, address EA9B), and 'end' (end of assembly, address EAEC8). Write down the label names and associated address values, as we will need to use them later.

2 JOYSTIK.HEX after 1st pass of WordStar edit

Normally a .COM (command) file would be created from the .HEX file by using the CP/M LOAD command. But in this application, we want just the data portion of the .HEX file to be used in this special application to interface MBASIC to the Osborne 1 computers machine code (i.e., the data!). So first, we will use WordStar to edit the JOYSTIK.HEX file in either document or non-document mode (we will clean-up the file with PIP later, since it will contain special control code left by WordStar that antagonizes MBASIC). Locate the start of the data field in each record, and enter RETURN (<cr> is used here to indicate the carriage return position) as follows:

```
:10EA8000
00F3D300AF3208EF3A002947D3013E012B
:10EA9000
3208EFFB78E61F3280EAC9F3D300AF32C9
:10EAA000
08EF3201293200293E04320129AF320336
:10EAB000
292F3202293E043203293EFF320229D394
:08EAC000
013E013208EFFBC921
:0000000000
```

3 JOYSTIK.HEX after 2nd pass of WordStar edit

Good, you have isolated the 'load information' from the data field ... now locate the end of the data field (one byte before each checksum) and enter RETURN as follows:

```
:10EA8000
00F3D300AF3208EF3A002947D3013E01
2B
:10EA9000
3208EFFB78E61F3280EAC9F3D300AF32
C9
:10EAA000
08EF3201293200293E04320129AF3203
36
:10EAB000
292F3202293E043203293EFF320229D3
94
:08EAC000
013E013208EFFBC9
21
:0000000000
```

4 JOYSTIK.HEX after 3rd pass of WordStar edit

Great. We are almost done, and you are finally finding out what those mysterious .HEX files were all about. Now position the WordStar cursor in front of each colon character and press ^ Y. Now carefully go back and delete each one byte checksum. When done, your original .HEX will look like this:

```
00F3D300AF3208EF3A002947D3013E01
3208EFFB78E61F3280EAC9F3D300AF32
08EF3201293200293E04320129AF3203
292F3202293E043203293EFF320229D3
013E013208EFFBC9
```

5 JOYSTIK.HEX after 4th pass of WordStar edit

Now that we have stripped the file to its bare essentials, we must enter a comma character to isolate each machine code data byte, one from the other, that will be required by MBASIC's READ statement. (Be sure you are in WordStar's INSERT mode (^V toggles insert ON and OFF) Do two character moves to the right (^D), enter a comma, two moves again, etc., etc. When you are done, the final (honest, we really *are* almost done!) .HEX should appear as follows:

```
00,F3,D3,00,AF,32,08,EF,3A,00,29,47,D3,01,3E,01
32,08,EF,FB,78,E6,1F,32,80,EA,C9,F3,D3,00,AF,32
08,EF,32,00,29,32,01,29,3E,04,32,01,29,AF,32,03
29,2F,32,02,29,3E,04,32,03,29,3E,FF,32,02,29,D3
01,3E,01,32,08,EF,FB,C9
```

6 Converting JOYSTIK.HEX to JOYSTIK.BAS

Now, enter ^K and D to save JOYSTIK.HEX, then use ^O to rename JOYSTIK.HEX TO JOYSTIK.BAS. With that done, edit JOYSTIK.BAS adding MBASIC line numbers (starting with some high number like 8000), and the statements required to block move the machine code into the area in the Osborne 1 system BIOS that normally is used for diskette data buffer transfers (this last comment was for the 'Techy-Types' only):

```
8000 DEFINT K   'double density (1.4x) code.  See note for others.
8010 DSTK=&HEA80:RSTK=&HEA81:ISTK=&HEA9B:H$="0123456789ABCDEF"
8020 FOR A=&HEA80 TO &HEAC7
8030 READ D$:K=(INSTR(H$,LEFT$(D$,1))-1)*16+INSTR(H$,RIGHT$(D$,1))-1
8040 POKE A,K:NEXT:RETURN
8050 DATA 00,F3,D3,00,AF,32,08,EF,3A,00,29,47,D3,01,3E,01
8060 DATA 32,08,EF,FB,78,E6,1F,32,80,EA,C9,F3,D3,00,AF,32
8070 DATA 08,EF,32,01,29,32,00,29,3E,04,32,01,29,AF,32,03
8080 DATA 29,2F,32,02,29,3E,04,32,03,29,3E,FF,32,02,29,D3
8090 DATA 01,3E,01,32,08,EF,FB,C9
Note: &HEA-- => 1.4x monitor rom.  (Also EA on line 8060.)
Change to &HED-- and ED on tenth byte on line 8060 => 1.3 mon rom
Change to &HEE-- and EE on tenth byte on line 8060 => 1.2 mon rom
```

Line 8000 defines integer variable K as the data value that will be used by the read statement in Line 8030 to "fetch" the data (your modified .HEX file) in Line 8050 to Line 8090. Line 8010 defines the addresses that you wrote down earlier. It performs the following:

- calls the subroutine to initialize the joystick (ISTK)
- reads the joystick (RSTK)
- calls an address from which we may PEEK at the data value read from the joy stick (DSTK)
- and finally calls a string constant H$ that defines the permissible range of hexadecimal digits (0 to F hex, or 1 to 15 decimal)

If you're using double density, the addresses are the same as the addresses shown here; if you're using 1.2 or 1.3 BIOS, they will have an offset of either 300 or 400; e.g., for 1.2 bios, EE9B instead of EA9B, EE81 instead of EA81.

Line 8020 is the statement to define the start of a FOR NEXT loop, where A is a counter whose first count is the "org" address (program origin EA80), and a final count of "end" minus one (EAC8-1 = EAC7). This is the block move address counter that will be used in Line 8040.

Line 8030 reads the data sequentially into string D$ from each DATA statement in Line 8050 to 8090, where the integer variable K is converted byte-by-byte to binary machine code (something the Z80 Central Processor Unit (CPU) in your Osborne 1 can actually execute directly without MBASIC's interpretive aid).

Line 8040 is simply a means to POKE into memory, the value in K pointed to by the address counter A. Just remember that A is the incrementing address starting at EE80, and K is the sequential read value of the DATA statements—just picked up from one place and put down (block moved) to another.

When the total FOR NEXT loop is completed (all data block moved), the MBASIC subroutine exits back to a (as yet unspecified) calling routine.

We are done with the basic (literally!) joystick routines, but just with the preliminary stuff. JOYSTIK.BAS may now be used as a library routine that you can use over and over again in various other MBASIC program applications that require a joystick interface. See you next time around with a joystick game for the Osborne 1.

In case the creation of cables and assembly language programs are a little beyond your capabilities or time requirements, you may contact Kelly Smith at P.O. Box 964, Chatsworth, CA 91311 to purchase the SPACE-WAR software (a 200 line program) and joystick hardware at a nominal fee. Indicate your computer's BIOS version (press the RESET button and look at the sign-on message).—bb

Kelly Smith is with the Wizard of OsZ, a software and accessories company. He was the author of the popular "Monster" program for the Portable Companion's inaugural issue.